hackathon3-regularization_ridge_regression

October 20, 2026

1 CSCI 447/547 Hackathon 3: Regularization, Ridge Regression, and LASSO

This notebook is designed to be started during class and continued as a take-home activity

1.1 How to use hackathon notebooks:

If the topic covered in a hackathon is new to you, work through the cells in order and read the explanation before running each code cell. You do not need to understand every detail of the hackathon on the first pass, instead, focus on the approach we take:

1. **Look at the data**
2. **Separate inputs from the target we want to predict**
3. **Split the data so we can test whether the model generalizes**
4. **Fit a model using the training data**
5. **Make predictions and evaluate them**
6. **Improve the model carefully without using the final evaluation data to make decisions**

We will work through the salary example together as a class. Pause before important code cells and think about what you expect to see. Please ask Lucy or a TA questions any time a term or line of code is unfamiliar.

After class, continue from wherever you left off. The existing explanations and code will be there to guide you through the process. Complete the marked answer sections, run every cell, and explain what the results mean in language that makes sense for you. Hackathons will not be graded, they are only to help you, and you will get out of them what you put into them.

The goal of this notebook is NOT to memorize every function. It is to identify the processes we use in machine learning and be able to reuse them.

In this exercise, you will implement Ridge regression LASSO, see how they work with data, and see the effects of regularization.

1.1.1 Local (VSCode/Jupyter Lab) Version

Make sure you

```
git pull
```

for the latest version of the repository and make sure that your `uv` virtual environment is enabled.

1.2 1. The Problem with Polynomials

In Hackathon 1, we implemented linear regression. However, real-world data rarely exhibits perfectly linear relationships. To capture non-linear trends, we can mathematically engineer new features by taking our existing inputs and combining them or raising them to a power (e.g., x_1^2, $x_1 x_2$).

While this makes our model more flexible, it introduces a severe risk: **Overfitting**. Today, we will look at how to control overfitting using mathematical penalties known as **Regularization**.

The Scenario: We will use the famous Boston Housing dataset to predict the median value of owner-occupied homes (`medv`).

[1]:
```
import numpy as np
import pandas as pd
import math

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_squared_error # Note that we'll be using MSE␣
 ↪for errors but this is something we could change

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
%matplotlib inline
```

And now lets import the data (depending on your envirnment/data location this cell will need to change).

[2]:
```
try:
    from google.colab import files
    uploaded = files.upload()

    train_df = pd.read_csv('train.csv', index_col='ID') # colab file import

except:
```

```
    train_df = pd.read_csv('data/boston-housing/train.csv', index_col='ID') #␣
 ↪local file import

train_df.info()
```

```
<class 'pandas.DataFrame'>
Index: 333 entries, 1 to 506
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   crim     333 non-null    float64
 1   zn       333 non-null    float64
 2   indus    333 non-null    float64
 3   chas     333 non-null    int64
 4   nox      333 non-null    float64
 5   rm       333 non-null    float64
 6   age      333 non-null    float64
 7   dis      333 non-null    float64
 8   rad      333 non-null    int64
 9   tax      333 non-null    int64
 10  ptratio  333 non-null    float64
 11  black    333 non-null    float64
 12  lstat    333 non-null    float64
 13  medv     333 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 39.0 KB
```

[3]:
```
train_df.head()
```

[3]:

ID	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
7	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.60	12.43	22.9

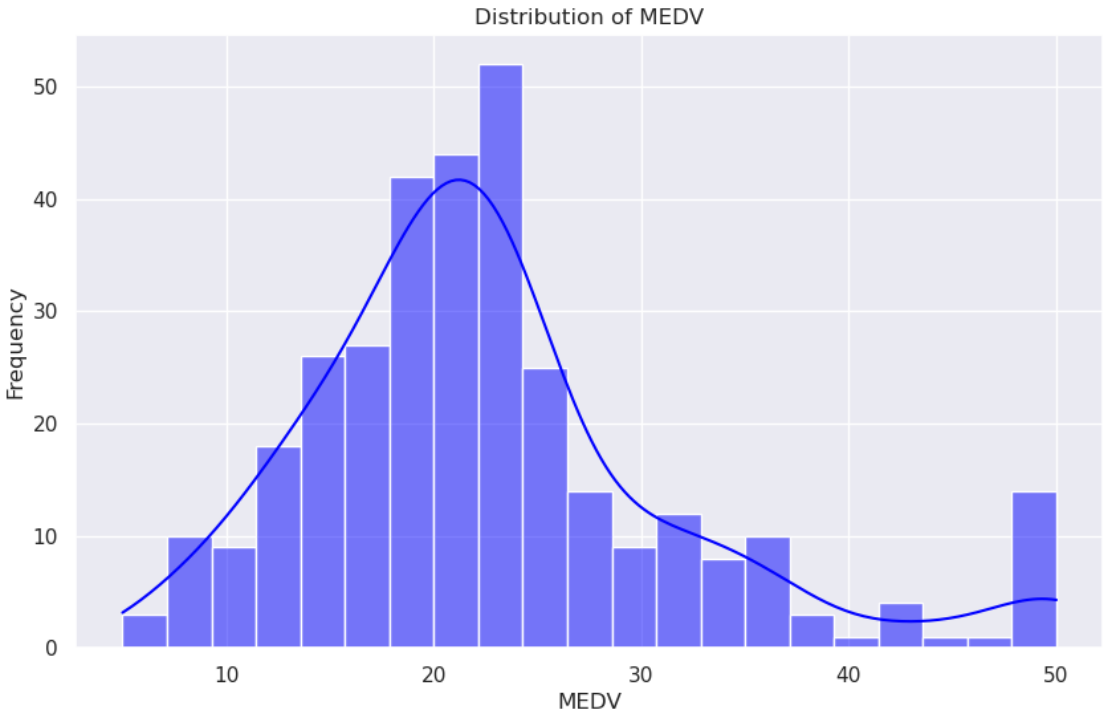
1.3 Data description

The Boston data frame has 506 rows and 14 columns.

Feature	Description
crim	Per capita crime rate by town.
zn	Proportion of residential land zoned for lots over 25,000 sq.ft.
indus	Proportion of non-retail business acres per town.
chas	Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).
nox	Nitrogen oxides concentration (parts per 10 million).
rm	Average number of rooms per dwelling.
age	Proportion of owner-occupied units built prior to 1940.
dis	Weighted mean of distances to five Boston employment centres.
rad	Index of accessibility to radial highways.
tax	Full-value property-tax rate per $10,000.
ptratio	Pupil-teacher ratio by town.
black	`1000(Bk - 0.63)^2` where `Bk` is the proportion of Black residents by town.
lstat	Lower status of the population (percent).
medv	Median value of owner-occupied homes in $1000s. *(Target Variable)*

Just to check out the code and see what we're dealing with, lets start by plotting the distribution of the median values of the homes (this is in $1Ks of dollars)

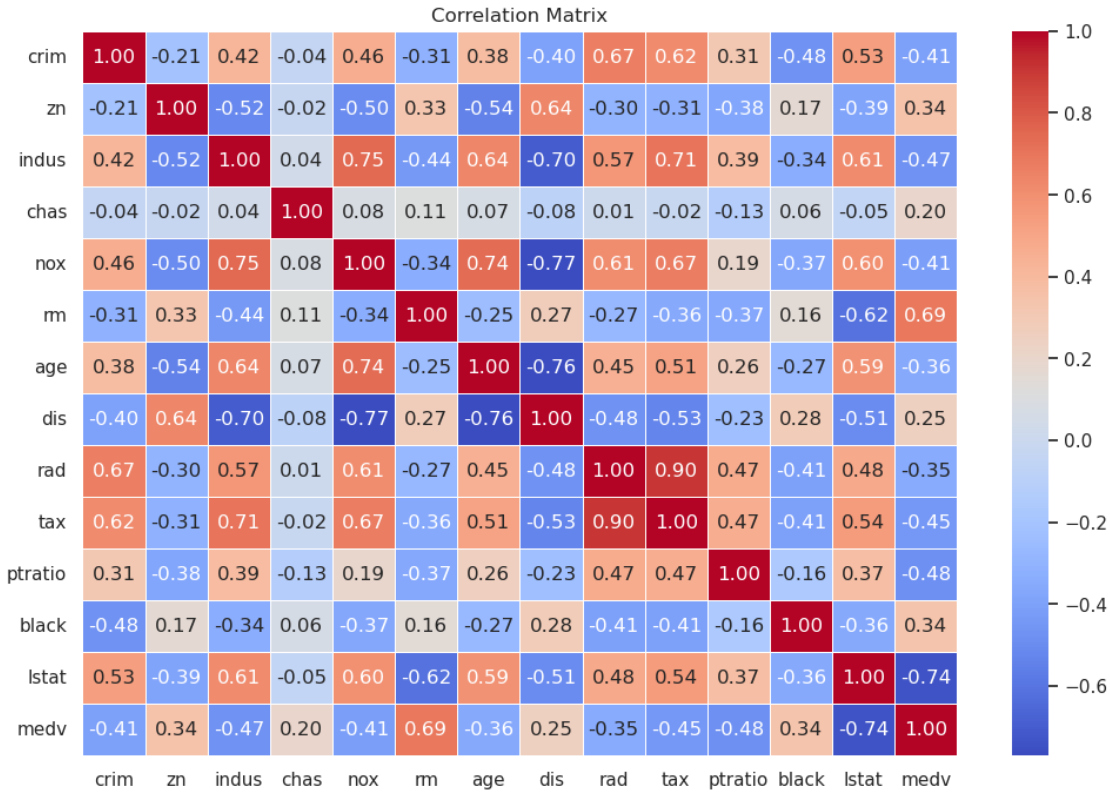
[4]:
```
# Distribution of MEDV
plt.figure(figsize=(10, 6))
sns.histplot(train_df['medv'], kde=True, color='blue')
plt.title('Distribution of MEDV')
plt.xlabel('MEDV')
plt.ylabel('Frequency')
plt.show()
```



And lets plot the correlation matrix for the data to see how medv is correlated with all these other features.

[5]:
```
# Correlation matrix
corr_matrix = train_df.corr()

# Visualizing the correlation matrix
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()
```



1.3.1 Think-Pair-Share 1: Feature Importance

Context: The correlation matrix above calculates the Pearson correlation coefficient (from -1 to 1) between all variables.

1. **Think:** Look specifically at the `medv` row/column (our target variable). Which single feature has the strongest *positive* correlation with home price? Which has the strongest *negative* correlation? Does this align with your logical intuition about real estate?

   Your individual hypothesis: Yes! Depending on the residental zone and the amount of rooms.

2. **Pair:** Discuss your findings with your group.

   Group consensus: Same.

3. **Share:** Be prepared to share your group's conclusion with the class.

1.3.2 1.2 Splitting the Data & Establishing a Baseline

First, we must isolate our target variable (`medv`), and split our data into training (70%) and testing (30%) sets. We will then fit a standard `LinearRegression` model to establish a baseline score.

[6]:
```
X = train_df.drop('medv', axis=1) ## The medv variable is the target variable,␣
 ↪so we'll drop it from the features
y = train_df['medv']



X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42,␣
 ↪test_size=0.3)
```

[7]:
```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

print('Training score: {}'.format(lr_model.score(X_train, y_train)))
print('Test score: {}'.format(lr_model.score(X_test, y_test)))

y_pred = lr_model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
# rmse = math.sqrt(mse)

print('MSE: {}'.format(mse))
```

```
Training score: 0.7268827869293253
Test score: 0.7254687959254529
MSE: 21.041489159411135
```

1.3.3 1.3 Feature Engineering: Polynomials

Our baseline model achieves an accuracy (R-squared) of ~72%. To improve this, we will engineer **Polynomial Features**. We will also apply a **StandardScaler**. As we learned in Hackathon 1, scaling is mathematically required so that larger features don't dominate the cost function.

We will use a scikit-learn `Pipeline` to chain these operations together cleanly.

[8]:
```
steps = [
    ('scalar', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2)),
    ('model', LinearRegression())
]

pipeline = Pipeline(steps)

pipeline.fit(X_train, y_train)

print('Training score: {}'.format(pipeline.score(X_train, y_train)))
print('Test score: {}'.format(pipeline.score(X_test, y_test)))
```

```
Training score: 0.9475767600691033
Test score: 0.46762684971882307
```

1.3.4 Think-Pair-Share 2: Diagnosing Model Failure

Context: Look at the output of the pipeline we just ran. Our training accuracy shot up to ~94%, but our test accuracy plummeted to ~46%.

1. **Think:** From an algorithmic perspective, what exactly did the model do during training that caused it to fail so spectacularly on unseen test data?

   Your individual hypothesis: The model is overfit to the training data.

2. **Pair:** Discuss this phenomenon (Overfitting / High Variance) with your group.

   Group consensus: High varience. The test score wildly varies from the traiing score. The new engineered feature is of higher order than the rest.

3. **Share:** Be prepared to share your group's conclusion with the class.

1.4 2. L2 Regularization (Ridge Regression)

To fix our overfitting problem, we will alter the Cost Function $J(\theta)$. During gradient descent, our model adjusted the weights (θ) to fit the training data perfectly. Ridge Regression artificially restricts this by adding a **penalty term** to the cost function.

$$J(\theta) = \text{MSE} + \alpha \sum_{j=1}^{n} \theta_j^2$$

This is the **L2 Norm penalty**. By adding the squared sum of the weights to the cost, the optimization algorithm is forced to keep the weights as small as possible. * If $\alpha = 0$, we just have standard Linear Regression. * As α increases, the penalty grows, and the model variance decreases (preventing overfitting).

You may also see this as λ, but sklearn uses ***alpha****.*

[14]:
```
steps = [
    ('scalar', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2)),
    ('model', Ridge(alpha=10, fit_intercept=True))
]

ridge_pipe = Pipeline(steps)
ridge_pipe.fit(X_train, y_train)

print('Training Score: {}'.format(ridge_pipe.score(X_train, y_train)))
print('Test Score: {}'.format(ridge_pipe.score(X_test, y_test)))
```

```
Training Score: 0.9181086448043114
Test Score: 0.8287450913722826
```

1.5 3. L1 Regularization (Lasso Regression)

Lasso (Least Absolute Shrinkage and Selection Operator) takes a slightly different approach. Instead of penalizing the *squared* weights, it penalizes the *absolute value* of the weights.

$$J(\theta) = \text{MSE} + \alpha \sum_{j=1}^{n} |\theta_j|$$

This subtle mathematical difference (L1 Norm vs L2 Norm) produces a radically different behavior during optimization.

1.5.1 Think-Pair-Share 3: Feature Selection

Context: Because of the geometry of the L1 penalty, Lasso tends to drive the weights of less important features to *exactly zero*. Ridge (L2) only shrinks them close to zero.

1. **Think:** Why might Lasso (L1) be highly preferred over Ridge (L2) if you are working with a dataset containing 10,000 features, but you suspect only 50 of them actually influence your target variable?

   Your individual hypothesis: The rest of the features might bog down computation.

2. **Pair:** Discuss the concept of "Feature Selection" and computational efficiency with your group.

   Group consensus: Don't have to derive 10,000 variables just for 50. Much more efficient.

3. **Share:** Be prepared to share your group's conclusion with the class.

---

[10]:
```
steps = [
    ('scalar', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2)),
    ('model', Lasso(alpha=0.3, fit_intercept=True))
]

lasso_pipe = Pipeline(steps)

lasso_pipe.fit(X_train, y_train)

print('Training score: {}'.format(lasso_pipe.score(X_train, y_train)))
print('Test score: {}'.format(lasso_pipe.score(X_test, y_test)))
```

```
Training score: 0.8483818643379695
Test score: 0.8307977758339726
```

1.5.2 4. Model Evaluation and Comparison

Notice the results of the Lasso pipeline: our training and testing scores are now much closer together (~85% and ~83%). By pushing irrelevant feature weights to zero, we successfully removed

the extreme variance (overfitting) of the polynomial model while maintaining strong predictive capability.

To formalize our analysis, we will build a programmatic evaluation framework. As we progress through the course and introduce new architectures (like Decision Trees and Support Vector Machines), we can plug them directly into this dictionary to compare their performance.

For this comparison, we will evaluate the models using **Mean Squared Error (MSE)**. While the R^2 accuracy score we printed above is useful for seeing the proportion of variance explained by the model, MSE provides an absolute measure of the average squared distance between our predictions and the true housing prices.

[11]:
```
# Function to evaluate a model
def evaluate_model(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    return mse

# Initialize the models
models = {
    "Linear Regression": pipeline,
    "Ridge Regression": ridge_pipe,
    "Lasso Regression": lasso_pipe,
}

# Evaluate each model
results = {}
for name, model in models.items():
    mse = evaluate_model(model, X_train, y_train, X_test, y_test)
    results[name] = {"Mean Squared Error": mse}

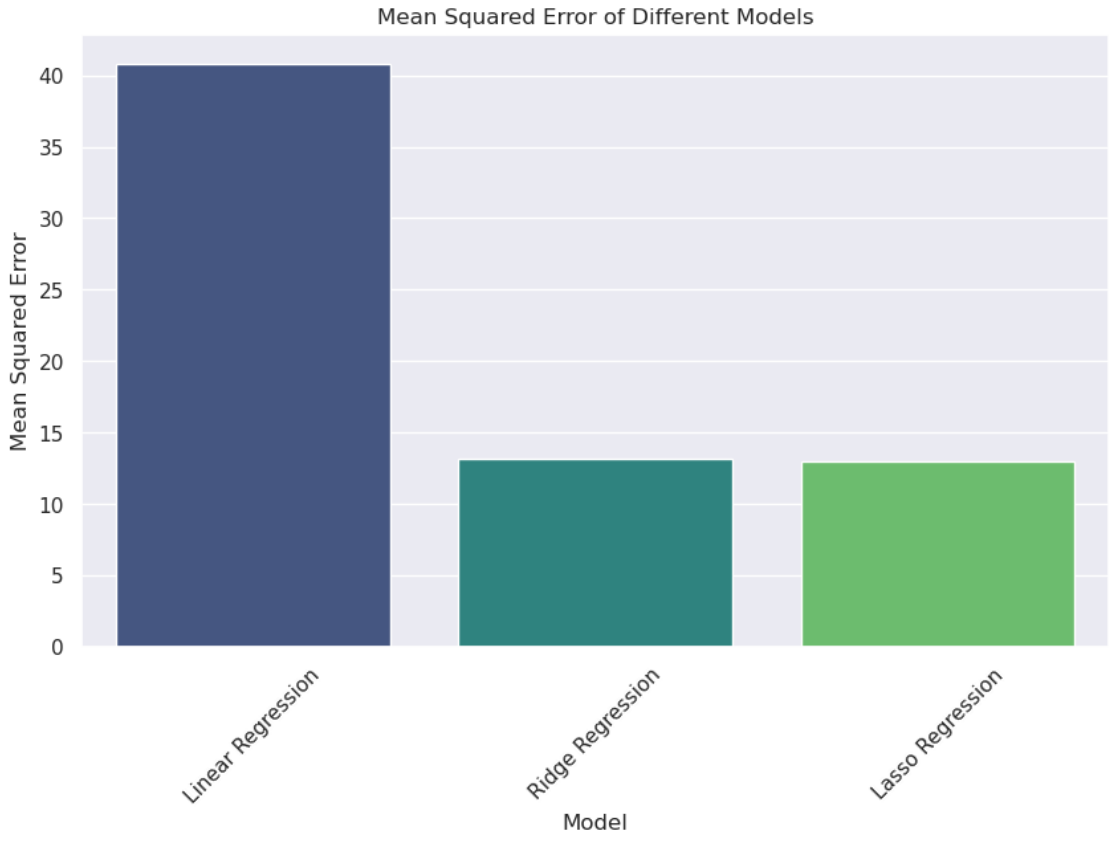
# Display the results
for name, metrics in results.items():
    print(f"{name} - Mean Squared Error: {metrics['Mean Squared Error']:.4f}")
```

```
Linear Regression - Mean Squared Error: 40.8038
Ridge Regression - Mean Squared Error: 13.1259
Lasso Regression - Mean Squared Error: 12.9685
```

[12]:
```
# Data for plotting
models = list(results.keys())
mse_values = [results[model]["Mean Squared Error"] for model in models]

# Plot Mean Squared Error
plt.figure(figsize=(10, 6))
sns.barplot(x=models, y=mse_values, hue=models, palette='viridis', legend=False)
plt.title('Mean Squared Error of Different Models')
plt.xlabel('Model')
```

```
plt.ylabel('Mean Squared Error')
plt.xticks(rotation=45)
plt.show()
```



1.6 Hackathon Takeaways & Synthesis

As your group finishes this notebook, collaborate to answer the following synthesis questions.

1. **Trade-off Analysis (Bias vs. Variance):** When we increased the complexity of our model using Polynomial features, we increased Variance (Overfitting). Regularization reduces Variance, but it increases Bias (Underfitting). If you set your Ridge penalty $\alpha = 1,000,000$, what do you expect your training and testing scores to look like?

   The models wights will approach zero. Making the model have extreme bias (underfitting).

2. **Model Architecture:** Why is it mathematically critical to place the `StandardScaler()` *before* the Ridge/Lasso models in our Pipeline? What would happen to the regularization penalty if we didn't scale the data first?

   If not standarization is applied, then the sensitivty beteen wights wildly varies. This can effect gradient decent.

3. **3-2-1 Summary:**

- **3** key mathematical or programming concepts your group solidified today:
    1. Regularization
    2. Abs vs Square
    3. corrilation table
- **2** things you are still slightly confused about (we will address these in the next lecture!):
    1. how to get that corrilation matrix
    2. Feature engineering
- **1** real-world scenario where you would explicitly choose Lasso (L1) over Ridge (L2):
    1.

Things you should be able to answer:

How are ridge and lasso regression similar? And how are they different? With respect to: The equations?

$$L_2 \text{ Regularization is MSE } + \alpha \sum \theta^2$$

$$L_1 \text{ Reg. is MSE } + \alpha \sum |\theta|$$

Interpretability of the model?

Because an L_2 regularization's θ^2 is a squared term. So the weight penalty becomes polynomal.

Can you interpret the model parameters (in class we've been calling these w's or weights, sklearn calls them coefficients so you'll want to use _coef)

Time? L_1 reg. has an absalute value term $|\theta|$ whose derivative at $|\theta| = 0$ is undefined because the absalute value funciton is discontinuous at that point. So there cannot be a closed form solution you have to preform grad decent.

While with the L_2 reg. you have that squared term θ^2 which is a continuous function so it has a closed form solution at $\theta = 0$.

Changing the value of lambda? if $\lambda = 0$ then the regularization term goes to zero, effectively removing it. Else if $\lambda \to \infty$

Collinearity within the data?

Can you compare across different train/test splits and plot the results?

Here's a hint: Use RepeatedKFold() and cross_validate() How do the coefficients change across these different cross validation Can we use this technique to find the regularization penalty hyperparamter (lambda)? Are there different regression techniques that you can try? ElasticNet for instance?

[ ]:

[ ]: